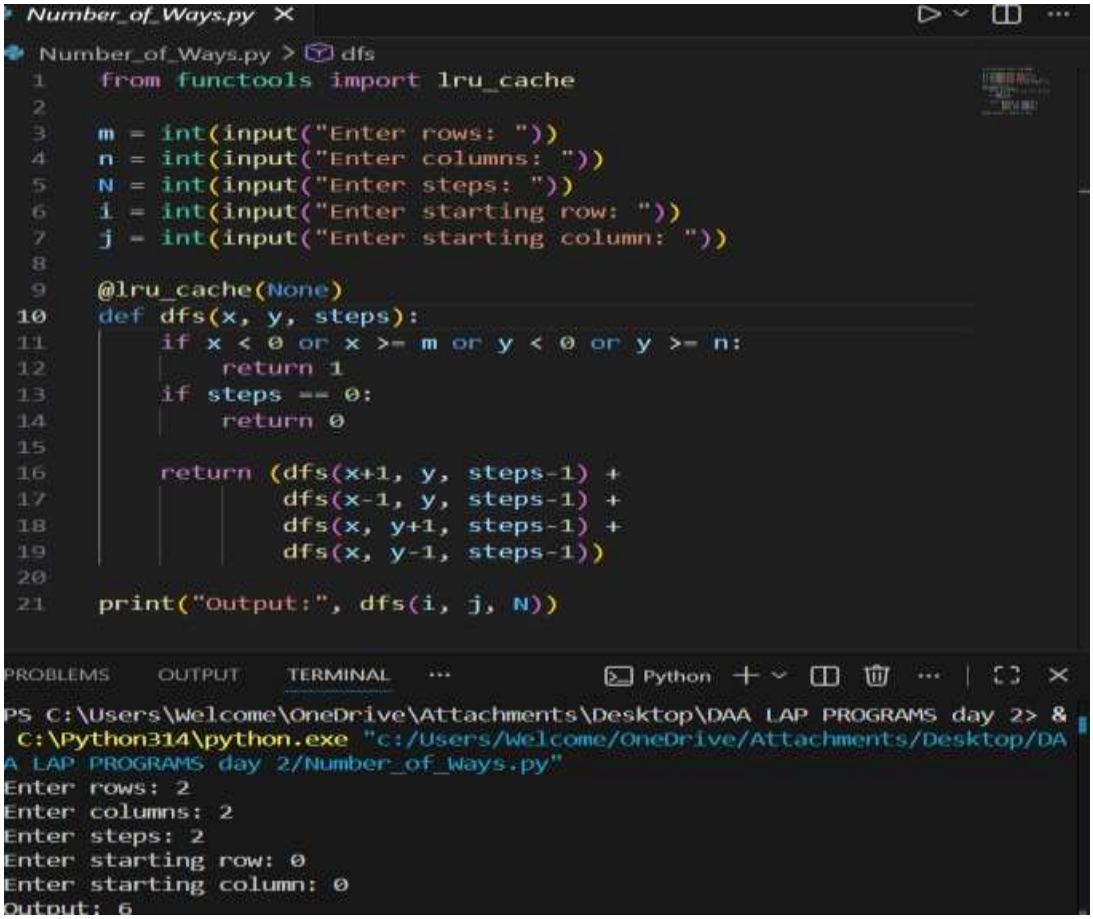
Name: Bestha Akshaya **Course code: CSA0611**
Regno:192424336 **Course name: DAA**

Slot C

1. **Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.**
CODE:

```
from functools import lru_cache

m = int(input("Enter rows: "))

n = int(input("Enter columns:
")) N = int(input("Enter steps:
"))
i = int(input("Enter starting row: "))

j = int(input("Enter starting column: "))



@lru_cache(None)
def dfs(x, y, steps):
   if x < 0 or x >= m or y < 0 or y >= n:
      return 1
   if steps == 0:
      return 0
   return (dfs(x+1, y, steps-1)
         + dfs(x-1, y, steps-1)
         + dfs(x, y+1, steps-1)
         + dfs(x, y-1, steps-1))


print("Output:", dfs(i, j, N))
```

OUTPUT:



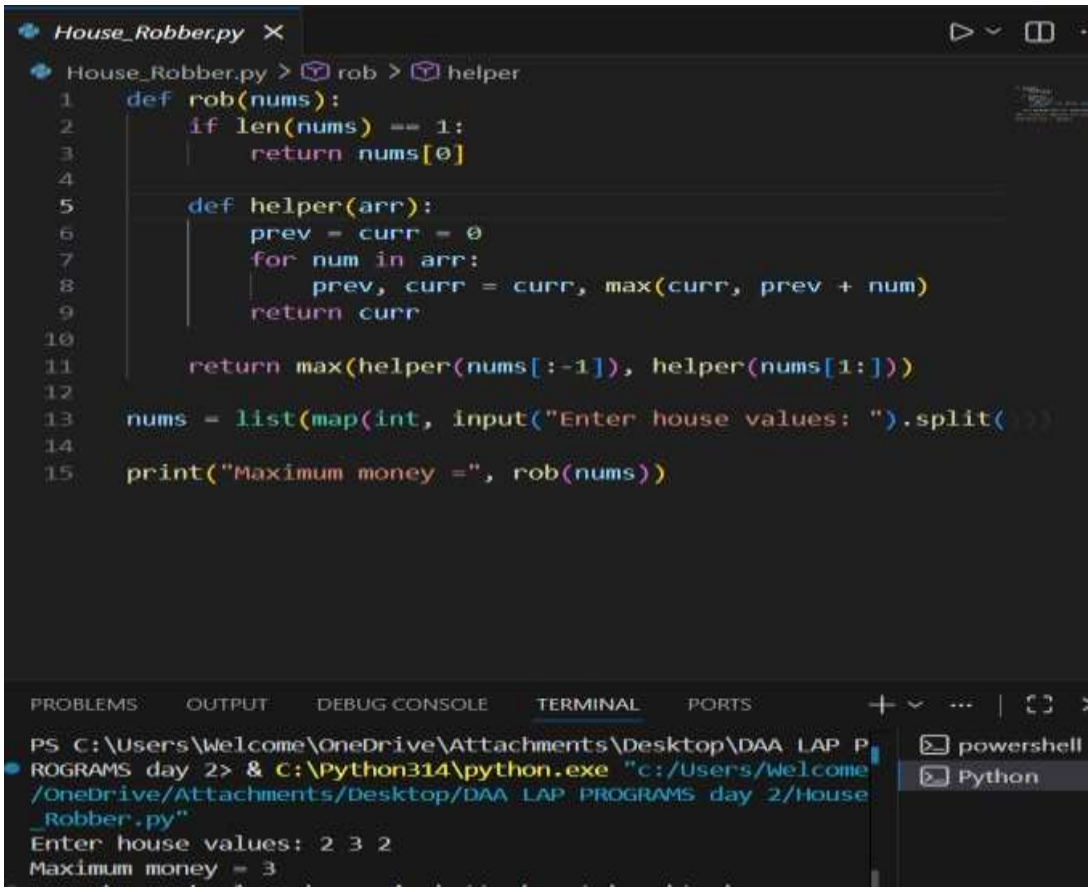
2. **You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.**

CODE:

```
def rob(nums):
    if len(nums) == 1:
        return nums[0]
    def helper(arr):
```

```
    prev = curr =
    0 for num in
    arr:
        prev, curr = curr, max(curr, prev +
    num) return curr
  return max(helper(nums[:-1]), helper(nums[1:]))

nums = list(map(int, input("Enter house values: ").split()))
print("Maximum money =", rob(nums))
```

OUTPUT:



3. **You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?**

Examples:

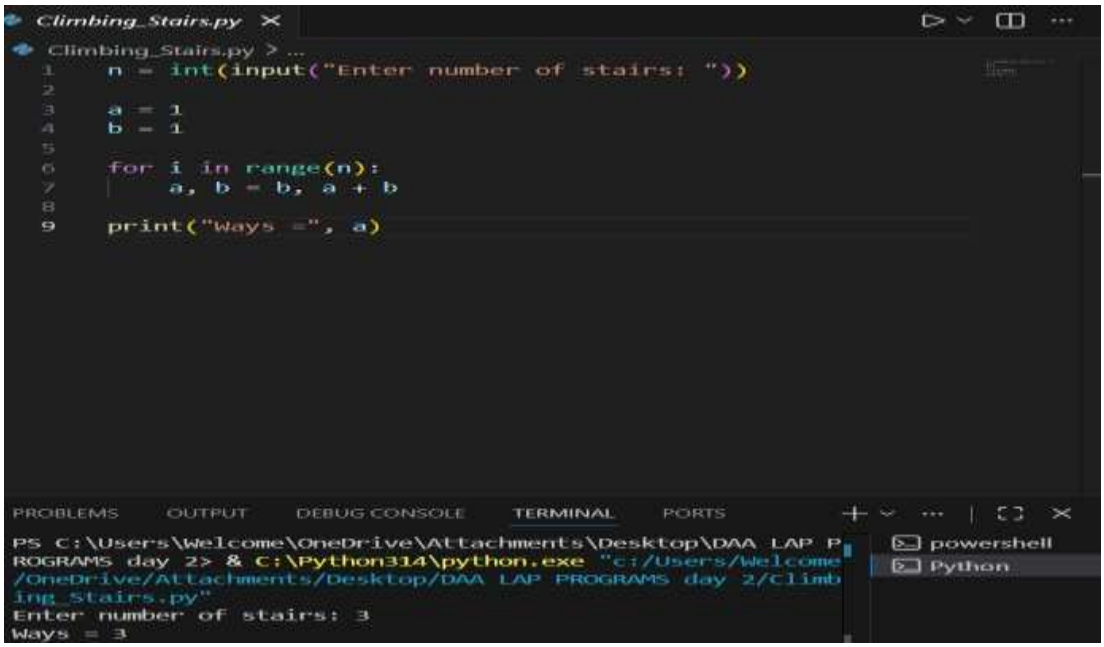
(i) Input: n=4 Output: 5

(ii) Input: n=3 Output: 3

CODE:

```
n = int(input("Enter number of stairs: "))

a = 1

b = 1

for i in
   range(n): a,
   b = b, a + b
print("Ways =", a)
```

OUTPUT:



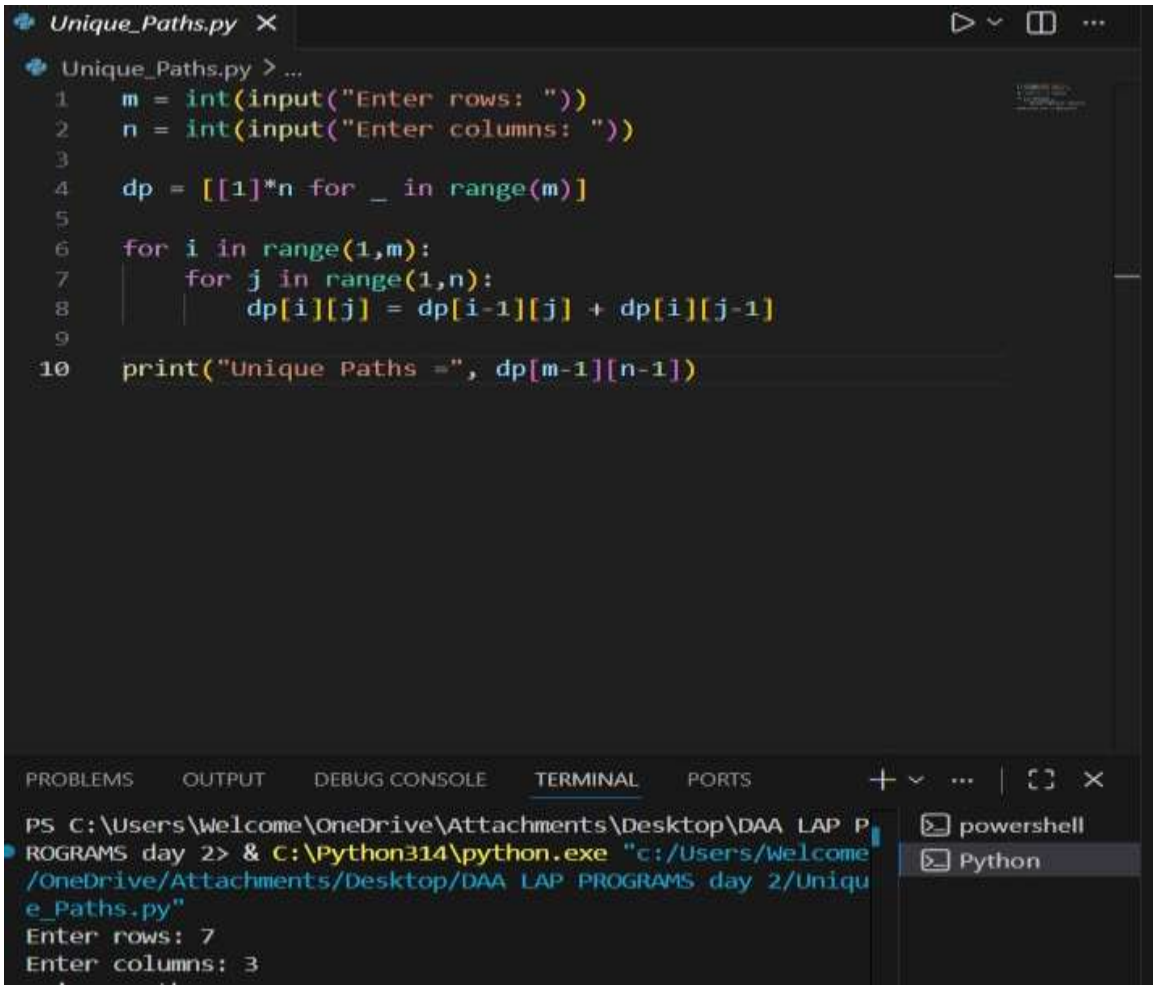
4. **A robot is located at the top-left corner of a m×n grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?**

CODE:

```
m = int(input("Enter rows: "))

n = int(input("Enter columns: "))
```

```
dp = [[1]*n for _ in range(m)] for i in range(1,m):
   for j in range(1,n):

dp[i][j] = dp[i-1][j] + dp[i][j-1]

print("Unique Paths =", dp[m-1][n-1])
```

OUTPUT:

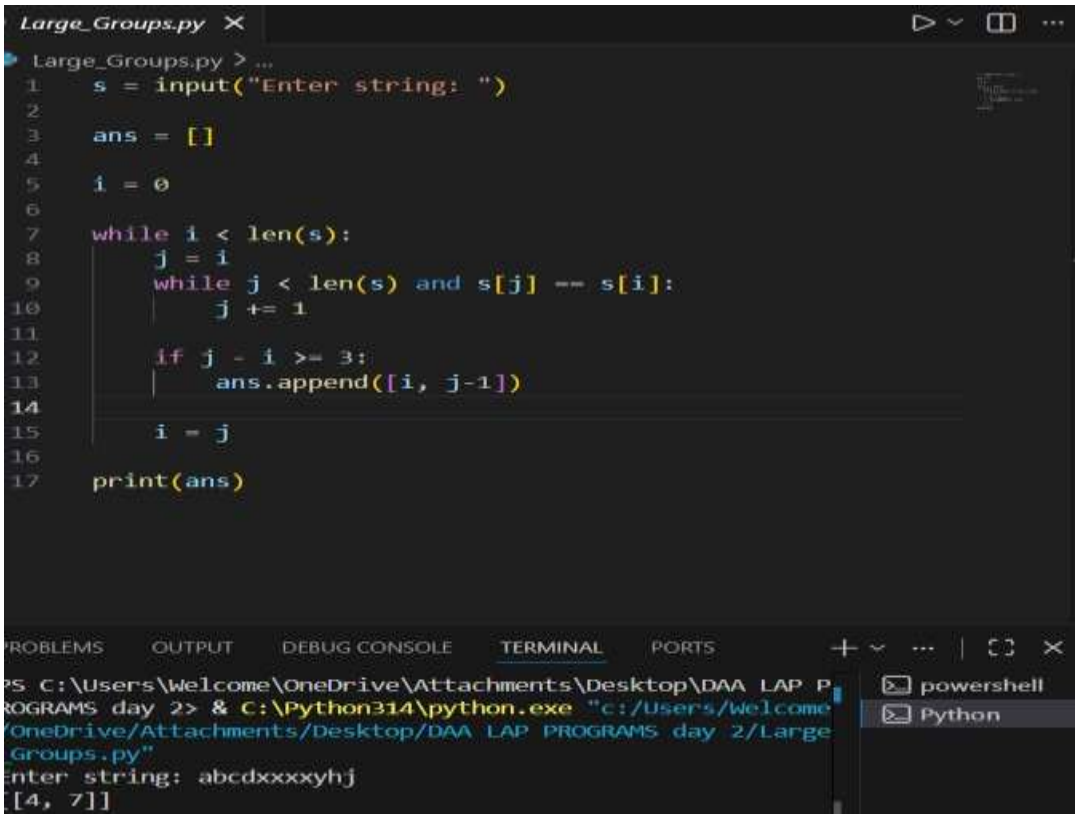


5. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the

intervals of every large group sorted in increasing order by start index.

CODE:

```
s = input("Enter string: ")
ans   =
[] i = 0
while     i     <
   len(s): j = i
   while j < len(s) and s[j] ==
      s[i]: j += 1
   if   j   -   i   >=   3:
      ans.append([i,   j-
      1])
   i   =   j
print(ans
)
```



6. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an m x n

grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

Any live cell with fewer than two live neighbors dies as if caused by under-population.

1. Any live cell with two or three live neighbors lives on to the next generation.

2. Any live cell with more than three live neighbors dies, as if by over-population.

3. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

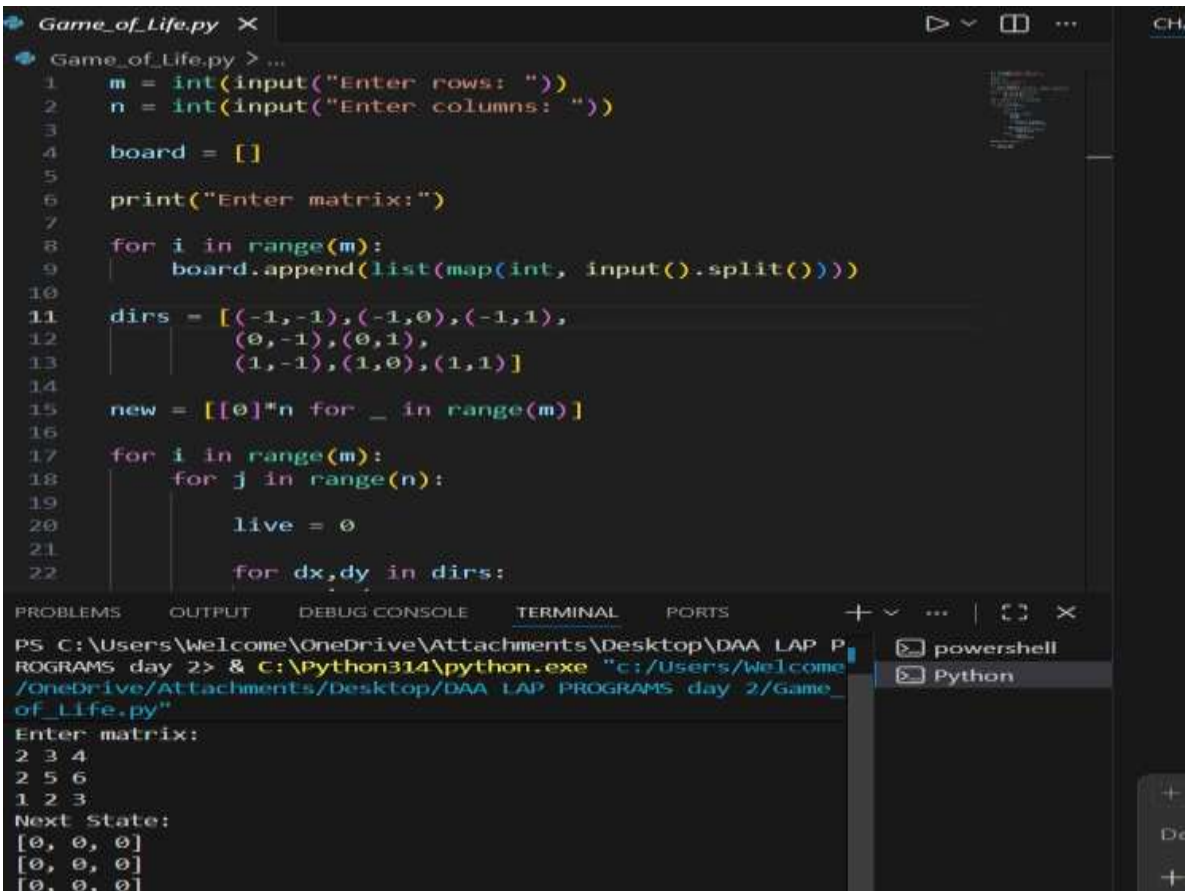
The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the m x n grid board, return the next state.

CODE:

```
m = int(input("Enter rows: "))

n = int(input("Enter columns:
")) board = []
print("Enter
matrix:") for i in
range(m):
   board.append(list(map(int,
input().split())))  dirs = [(-1,-1),(-1,0),(-1,1),
      (0,-1),(0,1),

      (1,-1),(1,0)

new = [[0]*n for _ in
range(m)] for i in range(m):
   for j in range
      live = 0
      for dx,dy in
```

```
            dirs: x=i+dx
            y=j+dy


            if      0<=x<m      and
               0<=y<n:   live   +=
               board[x][y]
         if board[i][j]==1:

            if  live==2  or  live==3:
               new[i][j]=1
         else:

            if live==3:


               new[i][j]
=1        print("Next
State:") for row in
new:
   print(row)
```

OUTPUT:


Game_of_Life.py
Game_of_Life.py > ...
m = int(input("Enter rows: "))
n = int(input("Enter columns: "))
board = []
print("Enter matrix:")
for i in range(m):
board.append(list(map(int, input().split())))
dirs = [(-1,-1),(-1,0),(-1,1),
(0,-1),(0,1),
(1,-1),(1,0),(1,1)]
new = [[0]*n for _ in range(m)]
for i in range(m):
for j in range(n):
live = 0
for dx,dy in dirs:
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Welcome\OneDrive\Attachments\Desktop\DAA LAP P
ROGRAMS day 2> & C:\Python314\python.exe "c:/Users/Welcome
/OneDrive/Attachments/Desktop/DAA LAP PROGRAMS day 2/Game_
of_Life.py"
Enter matrix:
2 3 4
2 5 6
1 2 3
Next State:
[0, 0, 0]
[0, 0, 0]
[0, 0, 0]
powershell
Python

7.We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess

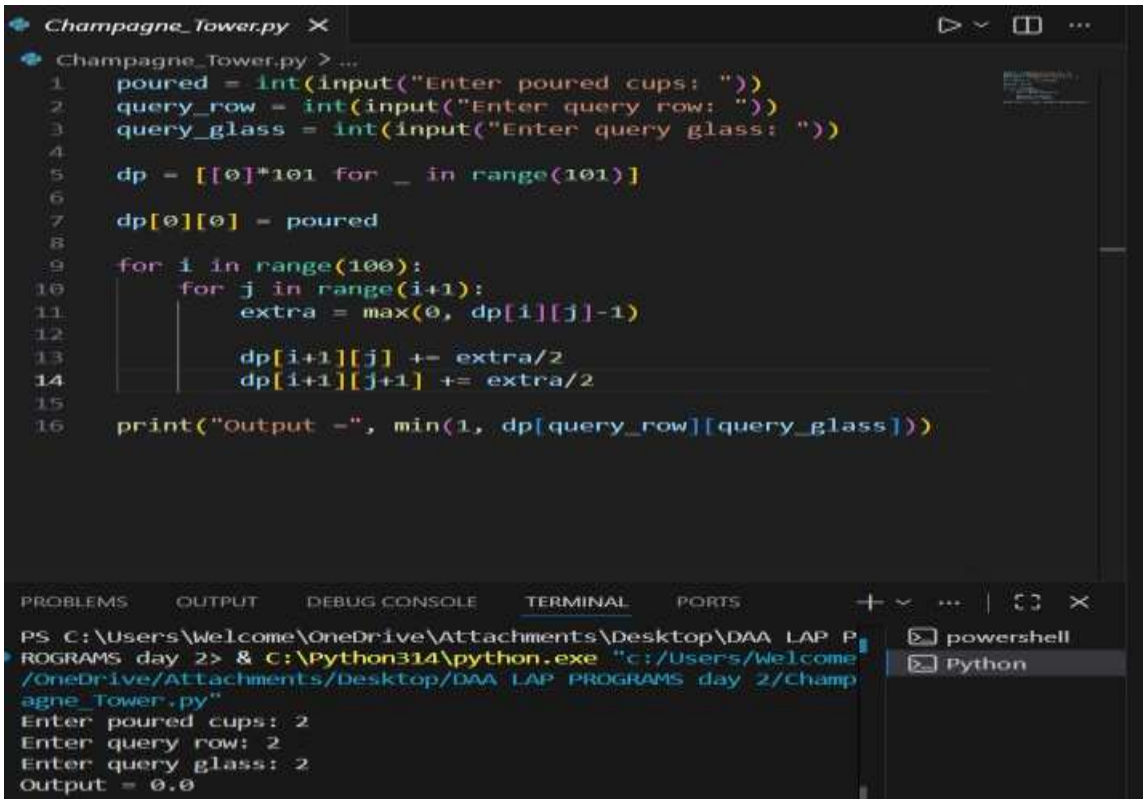
champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are

poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.

CODE::

```
poured = int(input("Enter poured cups: "))
query_row = int(input("Enter query row: "))
query_glass = int(input("Enter query glass:
")) dp = [[0]*101 for _ in range(101)]
dp[0][0] = poured
for i in
range(100):
   for j in range(i+1):

      extra = max(0, dp[i][j]-1)
      dp[i+1][j] += extra/2
      dp[i+1][j+1] += extra/2
print("Output =", min(1, dp[query_row][query_glass]))
```

OUTPUT:


Champagne_Tower.py
Champagne_Tower.py > ...
poured = int(input("Enter poured cups: "))
query_row = int(input("Enter query row: "))
query_glass = int(input("Enter query glass: "))
dp = [[0]*101 for _ in range(101)]
dp[0][0] = poured
for i in range(100):
for j in range(i+1):
extra = max(0, dp[i][j]-1)
dp[i+1][j] += extra/2
dp[i+1][j+1] += extra/2
print("Output =", min(1, dp[query_row][query_glass]))
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Welcome\OneDrive\Attachments\Desktop\DAA LAP P
ROGRAMS day 2> & C:\Python314\python.exe "c:/Users/Welcome
/OneDrive/Attachments/Desktop/DAA LAP PROGRAMS day 2/Champ
agne_Tower.py"
Enter poured cups: 2
Enter query row: 2
Enter query glass: 2
Output = 0.0
powershell
Python